

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4

по дисциплине «Программирование на языке Джава»

Выполнил студент группы ИКБО-70-25 **Мкртчян Г.Г.**

Принял Ермаков С.Р.
Старший преподаватель

Москва 2026

Задача 4.1.1

Описание задачи, цель

Создать класс Triangle, который наследуется от GeometricObject. Добавить три стороны со значениями по умолчанию 1.0, два конструктора, getter-методы, вычисление площади и периметра и метод toString(). Написать программу для ввода сторон, цвета и заливки. Цель: научиться создавать подклассы и использовать унаследованные свойства.

Ход работы

Создаем пакет org.example.pr4. В классе GeometricObject определяем цвет color, заливку filled и дату создания dateCreated. Добавляем методы для чтения и изменения свойств.

[Скриншот 1: GeometricObject.java — поля, конструкторы и методы доступа]

Создаем Triangle с полями side1, side2 и side3. Периметр находим как сумму сторон, площадь — по формуле Герона: s = (side1 + side2 + side3) / 2; area = √(s · (s − side1) · (s − side2) · (s − side3)). Метод toString() возвращает описание сторон.

[Скриншот 2: Triangle.java — конструкторы, getArea(), getPerimeter() и toString()]

В TestTriangle вводим стороны 3, 4, 5, цвет red и заливку true. После создания объекта выводим его свойства. Получаем площадь 6.0 и периметр 12.0.

[Скриншот 3: консоль TestTriangle — ввод 3 4 5, red, true и результат]

Результаты и выводы

Создали треугольник и программу для работы с ним. Научились использовать наследование, конструкторы и методы доступа. В итоговом коде также учтены доработки из следующих заданий.

Задача 4.1.2

Описание задачи, цель

Создать класс IllegalTriangleException. Изменить конструктор Triangle так, чтобы при нарушении неравенства треугольника выбрасывалось это исключение. Цель: научиться создавать и обрабатывать собственные исключения.

Ход работы

Создаем класс IllegalTriangleException, который наследуется от Exception. В сообщение передаем значения сторон, при которых не удалось создать треугольник.

[Скриншот 4: IllegalTriangleException.java — объявление класса и конструктор]

В конструкторе Triangle проверяем, что стороны положительные и сумма любых двух больше третьей. Если проверка не пройдена, выполняем throw new IllegalTriangleException(side1, side2, side3). В сигнатуру конструктора добавляем throws IllegalTriangleException.

[Скриншот 5: Triangle.java — проверка сторон и выбрасывание исключения]

В TestTriangle оборачиваем создание объекта в try и обрабатываем исключение в catch. Для проверки вводим стороны 1, 2, 3, цвет red и false. Сумма первых двух сторон равна третьей, поэтому получаем сообщение об ошибке. Стороны 3, 4, 5 проходят проверку.

[Скриншот 6: консоль TestTriangle — сообщение об ошибке для сторон 1 2 3]

Результаты и выводы

Добавили проверку существования треугольника. Научились использовать throw, throws и try-catch. При неверных сторонах объект не создается, а программа выводит понятное сообщение.

Задача 4.1.3

Описание задачи, цель

Сделать GeometricObject абстрактным классом, от которого наследуется Triangle. Реализовать Comparable и статический метод max(). Проверить сравнение двух кругов, двух прямоугольников, двух ComparableCircle, а также круга и прямоугольника. Цель: освоить абстрактные классы и сравнение объектов.

Ход работы

Объявляем GeometricObject абстрактным, а getArea() и getPerimeter() — абстрактными методами. В Triangle, Circle и Rectangle задаем свои формулы. Ввод сторон, цвета и заливки проверяем через TestTriangle.

[Скриншот 7: абстрактный GeometricObject и реализации методов в подклассах]

Добавляем Comparable<GeometricObject>. В compareTo() сравниваем площади с помощью Double.compare(). В max() возвращаем объект с большей площадью. При равенстве площадей возвращаем первый объект.

[Скриншот 8: GeometricObject.java — compareTo() и max(); ComparableCircle.java]

ComparableCircle наследуется от Circle и получает Comparable через базовый класс. Его compareTo() принимает GeometricObject, поэтому позволяет сравнить круг с прямоугольником. Запускаем TestComparison.

Для кругов с радиусами 2 и 3 выбирается радиус 3; для прямоугольников 3 × 4 и 4 × 5 — второй. Из ComparableCircle с радиусами 2 и 4 выбирается радиус 4. Между кругом радиуса 2 и прямоугольником 4 × 5 выбирается прямоугольник с площадью 20.0.

[Скриншот 9: консоль TestComparison — результаты четырех сравнений]

Результаты и выводы

Научились работать с абстрактными методами и интерфейсом Comparable. Реализовали единое сравнение фигур по площади и проверили max() и compareTo().

Задача 4.1.4

Описание задачи, цель

Создать интерфейс Colorable с методом howToColor(). Класс Square должен наследоваться от GeometricObject и реализовывать Colorable. Создать массив из пяти фигур, вывести площадь каждой и вызвать howToColor(), если фигуру можно раскрасить. Цель: закрепить работу с интерфейсами и полиморфизмом.

Ход работы

Создаем интерфейс Colorable с методом void howToColor(). В Square добавляем поле side, getter и setter, конструктор без параметров со стороной 0 и конструктор с указанной стороной.

[Скриншот 10: Colorable.java и Square.java — поле, конструкторы и методы доступа]

Площадь квадрата вычисляем как side * side, периметр — как 4 * side. Метод howToColor() выводит сообщение «Раскрасьте все четыре стороны.».

[Скриншот 11: Square.java — getArea(), getPerimeter() и howToColor()]

В TestColorable создаем массив GeometricObject[]: Square(2), Circle(3), Rectangle(3, 4), Square(5) и Square(). Для каждой фигуры выводим площадь. Через instanceof Colorable проверяем, можно ли вызвать howToColor().

Получаем площади 4.0, примерно 28.2743, 12.0, 25.0 и 0.0. Сообщение о раскрашивании выводится для трех квадратов.

[Скриншот 12: TestColorable.java и консоль — пять фигур и подсказки для квадратов]

Результаты и выводы

Создали интерфейс и реализовали его в классе Square. Научились хранить разные фигуры в одном массиве и вызывать дополнительные методы после проверки типа.

Задача 4.2.1

Описание задачи, цель

С помощью двух массивов написать программу, которая принимает целое число от 1 до 12 и выводит месяц и количество дней. Недопустимый номер обработать через ArrayIndexOutOfBoundsException. Предотвратить ввод значений, которые не являются целыми числами. Цель: научиться обрабатывать ошибки при работе с массивами и вводом.

Ход работы

Создаем MonthDaysBasic. В массив months записываем названия месяцев, в массив dom — количество дней. В базовой версии для февраля используем 28 дней.

[Скриншот 13: MonthDaysBasic.java — массивы months и dom]

Перед чтением номера проверяем scanner.hasNextInt(). Если введено слово или дробное число, пропускаем неверное значение через scanner.next() и предлагаем повторить ввод.

[Скриншот 14: MonthDaysBasic.java — проверка целого числа и блок try-catch]

Обращаемся к массивам по индексу month - 1. Если номер выходит за диапазон, перехватываем ArrayIndexOutOfBoundsException и выводим «Недопустимое число». Проверяем номер 3: получаем март и 31 день. При вводе 13 выводится сообщение об ошибке. При вводе abc программа просит целое число.

[Скриншот 15: консоль MonthDaysBasic — правильный номер, номер 13 и ввод abc]

Результаты и выводы

Научились сопоставлять данные двух массивов и перехватывать выход за их границы. Добавили проверку типа ввода, чтобы программа не принимала дробные числа и текст вместо целого номера.

Задача 4.2.2

Описание задачи, цель

Дополнить программу из предыдущего задания: при выборе февраля запросить год. Добавить метод для определения високосного года и изменить число дней в феврале. Цель: научиться выделять проверку в отдельный метод и использовать ее результат.

Ход работы

Создаем расширенную версию MonthDays. После ввода номера месяца проверяем, выбран ли февраль. Только в этом случае дополнительно запрашиваем целый положительный год.

[Скриншот 16: MonthDays.java — ввод месяца и запрос года для февраля]

Добавляем метод isLeapYear(int year). Год считается високосным, если делится на 400 или делится на 4, но не делится на 100. Условие: year % 400 == 0 || (year % 4 == 0 && year % 100 != 0).

[Скриншот 17: MonthDays.java — isLeapYear() и выбор между 28 и 29 днями]

Если isLeapYear() возвращает true, задаем 29 дней, иначе 28. Проверяем февраль 2024 года: получаем 29 дней; 2025 года — 28. Отдельно проверяем 1900 год — 28 дней и 2000 год — 29 дней. Для остальных месяцев ввод года не требуется.

[Скриншот 18: консоль MonthDays — февраль для 2024, 2025, 1900 и 2000 годов]

Результаты и выводы

Добавили расчет високосного года и корректное количество дней в феврале. Научились выносить повторяемую проверку в отдельный метод и учитывать исключения из общего правила делимости на 4.